



Caruca v2

LLM-based specification mining for opaque shell commands

System Architecture

How the existing and new pieces fit together

SOURCE `ai_docs/prep/system_architecture.md`

GENERATED `August 29, 2026 at 7:07 PM`

PAGES `7`

CARUCA V2 — RESEARCH PROJECT

Table of contents

System Architecture

Foundation (v1, existing) + Extensions (v2, new)

Risk Assessment

Open Architectural Decisions

System Architecture

Produced via the `09_generate_system_design` planning pass. Reads on from `ai_docs/prep/component_functionality.md` (per-component I/O and CLI surface) and `ai_docs/prep/data_telemetry_schema.md` (what gets recorded), and anchors the build-order pass next.

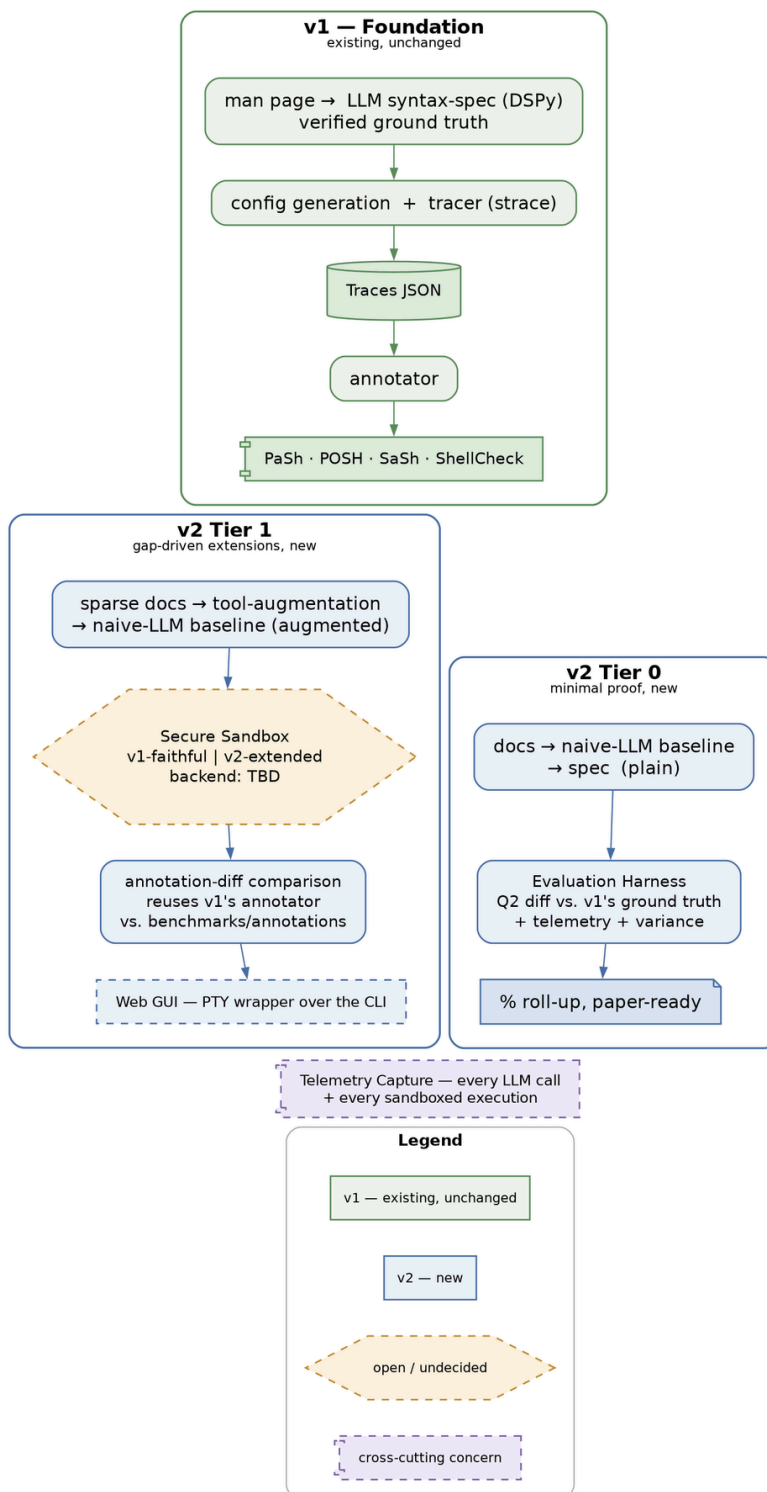
Foundation (v1, existing) + Extensions (v2, new)

Rendered diagram (letter-page, 300 DPI, with legend):

[ai_docs/diagrams/001_caruca_v2_system_architecture.png](#) · [SVG](#) · [GraphViz source](#) · [design rationale](#)

caruca_v2 System Architecture

Foundation (v1) + Extensions (v2) • Tier 0 = minimal proof, Tier 1 = gap-driven extensions



The same structure in text, for grep-ability and for readers who want the relationships spelled out rather than drawn:

v1 – FOUNDATION (existing, unchanged)

man page → LLM syntax-spec (DSPy) → config generation → tracer (strace)
 → Traces JSON ← the reusable trace/annotate seam
 → annotator → PaSh | POSH | SaSh | ShellCheck

v2 TIER 0 – MINIMAL PROOF (new; no sandbox required)

man page + docs → naive-LLM baseline (plain condition) → spec
 → Evaluation Harness: Q2 argument diff vs. v1's ground truth,
 plus telemetry and repeated-sampling variance
 → percent-change roll-up (paper-ready)

v2 TIER 1 – GAP-DRIVEN EXTENSIONS (new)

sparse-doc commands → LLM tool-augmentation → naive-LLM baseline (augmented condition)

v1's spec + naive-LLM's spec (both conditions) → Secure Sandbox
 (profiles: v1-faithful | v2-extended; backend still open)
 → v1's annotator, reused unmodified
 → annotation-diff comparison vs. benchmarks/annotations

real execution-based Q1 (PaSh/ShellCheck/Shseer suite reruns) – deferred,
 matches the paper's own Q1

Web GUI – a PTY wrapper that spawns the CLI subcommands above; presentation only

CROSS-CUTTING

Telemetry capture wraps every LLM call and every sandboxed execution.

Three structural properties are worth stating explicitly, because they are what make the comparison work:

- **v1 is extended, not displaced.** Every v1 stage stays in the picture and keeps running. v2's Secure Sandbox sits alongside v1's tracer for cases v1's model was not scoped to cover (destructive commands, undocumented binaries), and v2's annotation-diff path reuses v1's annotator rather than reimplementing it.
- **The naive-LLM baseline occupies the same pipeline slot as v1's own LLM step.** Because it emits the same Python-DSL spec shape, it feeds the identical downstream config-gen, trace, and annotate stages. That single choice buys both the Tier 0 syntax-

level comparison and the Tier 1 annotation-level one, and it means the sandbox and annotator are shared infrastructure rather than per-approach duplicates.

- **There is one front door, not two.** The Web GUI spawns the same CLI subcommands inside PTYs rather than calling a parallel API, so it cannot drift from what the CLI actually does.

Risk Assessment

Foundation strengths

- The trace/annotate boundary is a real JSON file (`Traces`), so tracing can run on a provisioned host while annotation and evaluation run anywhere. It is also the natural point to A/B two annotators on identical trace input.
- `cmp_specs.py` gives a reusable, already-published comparison method, and its ground truth is genuine: git history on `syntax_specs/*.py` shows 92 commits across 4 human authors with explicit “fix ground truths” / “update ground truths with types” curation, including a types pass matching the paper’s own reported discrepancy.
- Tier 0 needs none of the environment this machine lacks. Generation, annotation, spec-diffing, and analysis all run locally; only Tier 1’s sandbox work requires the provisioned host.

Integration points needing mitigation

- **Sandbox backend is unspiked.** Docker/overlayfs extension, Firecracker, and gVisor are all candidates, and at least gVisor’s ptrace compatibility is unverified. Because strace-level visibility is a hard constraint, this needs a spike before Tier 1 implementation, not during it.
- **Annotator extension has no regression safety net.** Extending `FSInteraction` and `to_annotation()`’s path-classification for symlink, permission-denied, and SIGPIPE trace shapes is real work, and no currently-passing test exercises annotation output (the annotation-class imports are among the broken ones in v1’s drifted test suite). Restoring a minimal annotation test before touching that logic is cheaper than debugging it afterward.
- **The DSPy migration is a prerequisite, not a footnote.** Baseline Instrumentation cannot capture telemetry from a step that dies at import. The migration also has a concurrency wrinkle: `dspy.settings.configure` rebinds the LM process-wide, so parallel runs at different models or seeds need serializing or per-process isolation.
- **One evaluation number needs a fix before it is quotable.** `cmp_specs.py`’s percentage divides a mismatched-element count by a dataclass-field count, so it does not reproduce

the paper's argument-level figure. The command-level exact-match tally is unaffected and safe to use as-is, which is why Tier 0 leads with that one.

● Decisions already made that reduce risk

- Deferring the agentic rebuild keeps the comparison honest: a deliberately naive prompt against an engineered pipeline, rather than two maximally-engineered systems.
- Reusing v1's ground truth and diff methodology keeps correctness numbers comparable to the published results instead of introducing a new, incomparable metric.
- Tiering means Tier 0 can ship and be evaluated without the sandbox backend question being answered at all, so the highest-uncertainty component blocks the least work.
- JSON sidecars as source of truth with SQLite as a derived query layer avoids a migration story during the phase when these schemas are still changing weekly.

Open Architectural Decisions

1. **Secure Sandbox isolation backend.** Docker/overlayfs extension vs. Firecracker vs. gVisor. Needs a ptrace-compatibility spike. Blocks Tier 1 only. Worth deciding with cross-platform portability in mind, since the paper motivates the problem partly with cross-platform variation while its stack is Linux-specific (`evaluation_gaps.md` gap #3).
2. **Extended Traces schema versioning.** Strict superset of v1's model vs. an explicitly versioned schema. Needs deciding before the v2-extended fixture fields land, since v1's own tracer and annotator read that JSON directly.
3. **Annotator extension scope.** How far to extend `FSInteraction` and the path-classification logic for v2-extended trace shapes, and whether mishandled shapes get reported as coverage-gap findings (the current plan) or fixed first.
4. **Web GUI timing.** Design is settled (PTY wrapper over the CLI); only its position in the build order is open, and it currently sits last.

Not open, recorded here to prevent re-litigation: the naive-LLM baseline emits v1's Python-DSL format; v2 wraps v1 as a subprocess rather than importing it as a library; telemetry lives in sidecar JSON with a derived SQLite layer; `sweep / compare` is a typed function with the CLI as a thin wrapper, which makes it callable by an LLM agent today without a bespoke tool integration.